مشروع E-commerce Orders Monitoring Pipeline :

هدف المشروع: بناء نظام بسيط لمحاكاة تدفق طلبات متجر إلكتروني، ومعالجة البيانات، وتوفير API للوصول إليها.

المشروع يتكون من **3 مراحل رئيسية**:

1- مرحلة1: توليد البيانات Data Generation.
2- مرحلة2: معالجة البيانات Data pipeline.
3- مرحلة3: الوصول للبيانات API Layer.

المرحلة 1: توليد البيانات (Data Generation)

الهدف: محاكاة نظام متجر إلكتروني يولد طلبات جديدة باستمرار.

قاعدة البيانات: إنشاء قاعدة بيانات في PostgreSQL باسم: raw_db

ثم إنشاء جدول لتخزين البيانات الخام.

- 1- إنشاء قاعدة البيانات raw_db
- 2- إنشاء جدول raw_orders

column	type
order_id	integer
user_id	integer
product_id	integer
quantity	integer
price	float
status	text
created_at	timestamp

- 3- إنشاء data generator script

اكتب سكربت Python يقوم بتوليد بيانات باستخدام

4- Faker & Random

field	description
order_id	رقم الطلب
user_id	رقم المستخدم
product_id	رقم المنتج
quantity	الكمية
price	السعر
status	حالة الطلب
created_at	وقت إنشاء الطلب

الحالات الممكنة: created- paid - shipped

- 5- إدخال البيانات إلىPostgreSQL

كل مرة يتم توليد سجل جديد يجب:

1. الاتصال بقاعدة البيانات
2. إدخال السجل في جدول orders

- 6- تشغيل السكربت كل ثانية

استخدم مكتبة: schedule لتنفيذ توليد الطلبات كل ثانية.

السلوك المطلوب: كل ثانية يتم إنشاء طلب جديد ويتم إدخاله في جدول orders

المرحلة 2: بناء Data Pipeline

الهدف: معالجة البيانات الخام وتحويلها إلى بيانات نظيفة

- 7- إنشاء قاعدة بيانات جديدة analytics
- 8- إنشاء جداول التحليل

أنشئ الجداول التالية:

جدول البيانات الخام: raw_orders

جدول البيانات النظيفة: orders_clean

- 9- بناء Pipeline

اكتب سكربت Python يقوم بالخطوات التالية:

كل **دقيقة واحدة**:

1- قراءة البيانات الجديدة من قاعدة بيانات raw

2- تحويل البيانات إلى **DataFrame باستخدام Pandas**

3- تنظيف البيانات:

a. التأكد أن الكمية > 0

b. السعر > 0

c. الحالة ضمن القيم المسموحة

d. تحويل timestamp إلى datetime

4- إدخال البيانات إلى قاعدة البيانات: analytics.raw_orders و analytics.orders_clean

- 10- جدولة الـ Pipeline

استخدم: schedule لتشغيل الـ pipeline كل دقيقة.

المرحلة 3: إنشاءAPI

الهدف: تمكين المستخدمين أو التطبيقات الأخرى من الوصول إلى بيانات الطلبات المخزنة في قاعدة البيانات من خلال API.

سيتم استخدام:

5- FastAPIلإنشاء API سريع.

6- Pydanticللتحقق من صحة البيانات.

7- SQLAlchemy للتعامل مع قاعدة البيانات.

- 11- إنشاء مشروع FastAPI

إنشاء API باستخدام: FastAPI - Pydantic- SQLAlchemy

الاستخدام	الأداة
إنشاءAPI	FastAPI
التحقق من البيانات	Pydantic
الاتصال بقاعدة البيانات	SQLAlchemy
تخزين البيانات	PostgreSQL

12- إنشاء Endpoint لآخر الطلبات

إنشاء:API endpoint

GET /orders/latest

يعرض:

آخر 10 طلبات.

13- إحصائيات الطلبات

إنشاء:endpoint

GET /orders/stats

يعرض:

- عدد الطلبات في آخر ساعة
- مجموع المبيعات
- عدد الطلبات حسب الحالة

مثال الاستجابة:

```
{
        "orders_last_hour": 120
        "total_sales": 5430,
        "status_distribution": {
                "created": 40,
                "paid": 50,
                "shipped": 30
        }
}
```

في مشروع الـ API باستخدام FastAPIيتم تقسيم الكود إلى عدة مكونات لتنظيم المشروع

1. Main: الملف الرئيسي للتطبيق.
   - المهام:
     1. إنشاء تطبيق.FastAPI
     2. تسجيل الـ routers.
     3. تشغيل الـ.API
   - نقطة بدء تشغيل التطبيق.

2. Database: المجلد المسؤول عن الاتصال بقاعدة البيانات باستخدامSQLAlchemy
   - المهام:
     1. إنشاء اتصال بقاعدة البيانات.
     2. إنشاء Session للتعامل مع البيانات.

3. توفير دالة تستخدمها الـ endpoints للوصول إلى قاعدة البيانات.
- إدارة الاتصال بقاعدة البيانات.

3. Models: يحتوي على نماذج الجداول في قاعدة البيانات باستخدامSQLAlchemy
- المهام:
  1. تعريف بنية الجدول.(columns)
  2. ربط الكود بالجداول فيPostgreSQL
- تمثيل جداول قاعدة البيانات داخل الكود.

4. Schemas: نماذج البيانات باستخدامPydantic
- المهام:
  1. تحديد شكل البيانات في الطلب أو الاستجابة.
  2. التحقق من صحة البيانات.
- تحديد شكل البيانات التي يتعامل معها الAPI

5. Routers: المجلد الذي يحتوي API endpoints
- المهام:
  1. تعريف المسارات مثل:
     - GET /orders/latest
     - GET /orders/stats
  2. تنفيذ العمليات المطلوبة على البيانات.
- منطق الـ API والمسارات.

Suggested Files Structure:

```
project/
├── generator/
│    ├──── generate_orders.py
├── pipeline/
│    ├──── pipeline.py
│    ├──── db.py
├──── api/
│    ├──── main.py
│    ├──── database/
│    │    └──── database.py
│    ├──── models/
│    │    └──── order.py
│    ├──── schemas/
│    │    └──── order_schema.py
│    └──── routers/
│         └──── orders.py
└── requirements.txt
```